# HotelSpace – Development Plan

Project: HostelSpace — Hostel Management System

Team 11 • PES University, Bangalore
UE24CS341A : Software Engineering
Prepared 5 September 2026 • Planning version 1.0

Post-Deliverable 2 team handoff: Jira backlog, architecture, delivery process and implementation sequence.

| Planning decision | Baseline |
|---|---|
| Project identity | The requested cover title uses “HotelSpace”. The application, repository and project remain HostelSpace; this is not a product rename. |
| Technology decision | Next.js frontend + Python/Django REST Framework backend, following the latest request. This supersedes the earlier Django-templates-only proposal. |
| Schedule assumption | No faculty deadline or availability was supplied. Proposed six-week window: 7 September–18 October 2026. Target release: 16 October; 17–18 October reserved as buffer. These are not official due dates. |
| Implementation status | Synopsis and SRS PDFs have been prepared. No working code, live Jira board, configured CI pipeline or completed submission has been verified in this task. |

## Completed documents and remaining work

Deliverable 1: Synopsis with the problem, scope, intended users, features, Agile approach and functional ownership. Deliverable 2: five-page SRS with FR-01–FR-11, NFR-01–NFR-05, SEC-01–SEC-06, business rules, use cases, an ER model, field requirements and an initial traceability matrix. “Prepared” does not mean faculty-approved or submitted.

Remaining: baseline corrections; Jira/Git setup; architecture and detailed design; validation specification; all feature implementation; reviews, unit/integration/system testing and coverage; static/security checks; Docker and CI/CD evidence; deployment; system validation; backup/maintenance; final report, 15-minute presentation and individual demonstration.

# How to use this plan

| Section | What the team can use |
|---|---|
| 1. Jira Board Plan | Six epics, 24 complete stories, acceptance criteria, owners, points, dependency and traceability references. |
| 2. Tech Stack & Architecture Explanation | One selected stack, role of each tool, request flow and integration decisions. |
| 3. CI/CD Pipeline Explanation | PR gates, staging-to-production sequence, sample YAML and required deployment setup. |
| 4. Folder Structure (mapped to who owns what) | Monorepo tree, module owners and shared-file review rules. |
| 5. GitHub Workflow / Team Account Usage Plan | Individual accounts, branches, reviews and meaningful dated commits. |
| 6. Prompts to Give Antigravity (per module) | Six module prompt packs with story-specific scope, paths and acceptance requirements. |
| 7. Timeline / Build Sequence | Dated weekly work, dependencies, exit criteria and final release buffer. |

## Baseline corrections before coding

• The existing SRS still describes Next.js with direct SQL access. Revise its environment/interface sections to Next.js → Django REST API → database, record Django/DRF, testing and CI/CD, and update the synopsis technology section too. This PDF does not edit those earlier files.

• The SRS diagram reuses UC-02 for notices/rules and study-room status. Proposed correction: keep UC-02 for information and assign UC-11 to study-room status in the diagram, FR-03 mapping and RTM together. Until that revision is accepted, preserve the legacy mapping in Jira.

• The earlier ER drawing is a high-level model, not a complete implementation schema. Detailed design must add the omitted notice/rule, study-room, attendance-session, coupon and fee-due/payment-attempt details and all real constraints.

• Define the still-open parameters: permitted profile fields, study-room updater roles, attendance time/radius/accuracy policy, payment provider access and the NFR-01 test load. Proposed choices here require team review, not silent changes to approved requirements.

Scope stays fixed: no special stay, room booking, student laundry selection, continuous tracking, meal ordering or full accounting. Docker remains a small required evidence task because the supplied course guidelines called for containerization; omitting it needs faculty approval. AI-assisted work must be understood and reviewed, with truthful prompt records where required.

# 1. Jira Board Plan

Create a Scrum project named HostelSpace, key HOST if available. Add all four people using their own accounts. EP-01–EP-06 and US-01–US-24 below are planning IDs, not issued Jira keys; keep them in descriptions and add the actual HOST keys after creation.

## Board setup and issue creation order

• Create six Epics from the next pages. Create four Stories under each Epic and set their Epic/Parent field. EP-05 and EP-06 contain developer/evaluator stories for technical and delivery outcomes; implementation steps underneath remain Tasks or Subtasks.

• Keep the backlog as the planning area, not a duplicate workflow status. Use To Do → In Progress → Code Review → Testing → Done. Flag blocked issues instead of treating Blocked as a completed state.

• For every story copy the provided title, story, description and criteria. Set one Assignee, priority, estimate and labels; use the component when the project supports it, otherwise use a label.

• Add dependency links using “blocks/is blocked by”. Create six one-week sprints for the proposed dates in Section 7, but start only the current sprint. All estimates and dates are provisional team planning values.

• Use a Bug only for an observed defect: include the affected FR/story, steps, expected/actual result, environment and evidence. Do not create empty bugs to inflate the backlog.

## Story subtasks and completion rule

Use 3–5 meaningful subtasks where needed: schema/API; page and component; permissions/edge cases; automated tests; review and validation evidence. Keep the functional owner accountable for the complete frontend-to-database feature. Avoid twelve tiny administrative subtasks for a simple status card.

Definition of Done: acceptance criteria demonstrated; input and object permissions checked; migrations included; tests and required lint/security/build jobs pass; one independent PR review; merged code and evidence linked; RTM updated. A stub, screenshot or AI-generated file alone is not completion.

## Workload balance

| Owner | Stories | Points | Responsibility |
|---|---|---|---|
| Anvi Srinivas | 6 | 26 | Access, information, Director; architecture and handoff |
| Arin Singh | 6 | 29 | Leave, attendance, deviations; schema and validation |
| Aryan Singh | 6 | 27 | Study room and laundry; Docker and integration |
| Atiksh Gour | 6 | 29 | Complaints, mess, fees; CI/CD and maintenance |

Total: 24 stories / 111 initial points. Points measure relative effort, not hours, grades or required commit counts. Re-estimate at planning and split an 8-point story if it cannot fit the team’s actual capacity.

## EP-01 — Student Access and Hostel Information

Epic lead: Anvi Srinivas

Deliver secure sign-in, student profiles, hostel information and Director oversight. Anvi owns each feature from the Next.js screen through its Django API and tests; other members integrate through the shared permission contract.

| **Title** | US-01 — Sign in, sign out and open the correct dashboard |
|---|---|
| **Story** | As a hostel user, I want to sign in and out of my role dashboard, so that I can safely use the functions assigned to me. |
| **Description** | Use Django authentication and DRF session authentication behind the same-origin API proxy. Build the Next.js login form and role-aware navigation without duplicating authorization in the frontend. Apply CSRF protection to login and all state-changing requests, and invalidate the session on logout. |
| **Acceptance Criteria** | • An active account with valid credentials reaches its permitted dashboard; invalid or inactive accounts are denied.<br>• Protected APIs reject missing sessions and disallowed roles even if a URL is entered directly.<br>• Logout invalidates the session; a subsequent protected request is denied.<br>• Session cookies are HttpOnly and Secure in HTTPS deployment; missing or invalid CSRF tokens are rejected on writes. |
| **Story Points** | 5 |
| **Assignee** | Anvi Srinivas |
| **Priority** | High |
| **Labels/Component** | frontend,backend,auth; Accounts |
| **Requirement / Use Case** | FR-01; UC-01; SEC-01,02,03 |
| **Dependencies** | US-17, US-18 |

| **Title** | US-02 — View and update my student profile |
|---|---|
| **Story** | As a student, I want to view my hostel profile and correct permitted details, so that my contact information stays current. |
| **Description** | Provide a profile page and a DRF serializer tied to the signed-in user. In V1 the student can edit phone number; name, SRN, hostel, room and role remain staff-managed fields. Validate input on the server and show field-specific errors in the form. |
| **Acceptance Criteria** | • A student sees only their own profile; another student record cannot be read or changed.<br>• A valid phone update persists after a refresh.<br>• Empty or malformed phone input is rejected with a field message.<br>• Attempts to change SRN, room, role or another protected field are rejected or excluded from the permitted update. |
| **Story Points** | 3 |
| **Assignee** | Anvi Srinivas |
| **Priority** | High |
| **Labels/Component** | frontend,backend,db; Accounts |
| **Requirement / Use Case** | FR-01; UC-01; SEC-03,04 |
| **Dependencies** | US-01 |

## EP-01 — Student Access and Hostel Information (continued)

| Title | US-03 — Publish and read hostel notices, contacts and rules |
|---|---|
| **Story** | As a student, I want to read current hostel notices, contacts and rules, so that I can find reliable hostel information in one place. |
| **Description** | Create notice, contact and rule records and the corresponding read screens. The Hostel Director and explicitly authorized content staff may maintain these records through protected endpoints. Keep rule identifiers stable because deviation records refer to them. |
| **Acceptance Criteria** | • Students see active notices whose active-date window includes the current time.<br>• Authorized content staff can create, edit and deactivate information; students cannot.<br>• Missing notice titles and invalid active-date ranges return validation errors.<br>• Rules and hostel contacts remain readable; a referenced rule is deactivated rather than deleted. |
| **Story Points** | 5 |
| **Assignee** | Anvi Srinivas |
| **Priority** | Medium |
| **Labels/Component** | frontend,backend,db; Information |
| **Requirement / Use Case** | FR-02; UC-02; SEC-04 |
| **Dependencies** | US-01, US-18 |

| Title | US-04 — Manage authorized users and view Director summaries |
|---|---|
| **Story** | As a Hostel Director, I want to manage authorized accounts and view hostel summaries, so that I can supervise the system without editing unrelated source records. |
| **Description** | Build a Director-only dashboard rather than exposing Django superuser access to all staff. Show counts for pending leave, attendance by session, open complaints and unpaid fee records using the owning modules. Manage account activation, role assignment and approved settings through validated APIs. |
| **Acceptance Criteria** | • Non-Director users cannot access the Director management endpoints.<br>• The Director can activate or deactivate accounts and assign permitted roles; deactivated accounts lose access.<br>• Only one active Chief Warden is allowed by the agreed role-management rule.<br>• Summary counts match seeded source records; empty datasets show zero rather than an error. |
| **Story Points** | 5 |
| **Assignee** | Anvi Srinivas |
| **Priority** | Medium |
| **Labels/Component** | frontend,backend,reports; Director |
| **Requirement / Use Case** | FR-11; UC-10; BR-01 |
| **Dependencies** | US-01; US-06, US-07, US-14, US-16 for summaries |

## EP-02 — Leave, Attendance and Deviation Records

Epic lead: Arin Singh

Deliver the student-to-warden leave workflow, one-time location attendance and deviation totals. Keep multiple wardens and one Chief Warden, with server-side ownership checks and clear status changes.

| | |
|---|---|
| **Title** | US-05 — Submit and track a leave request |
| **Story** | As a student, I want to submit leave dates and a reason, so that my assigned warden can review my request. |
| **Description** | Create a leave record linked to the student and assigned warden. Use a Next.js form and a DRF endpoint with date validation and a visible Pending state. Record submission time and expose only the student's own request history. |
| **Acceptance Criteria** | • Valid dates and a non-empty reason create one request with an ID and Pending status.<br>• An end date before the start date or missing required input is rejected.<br>• The request stores the assigned warden and is visible to that warden.<br>• Students can read their own history but cannot read another student's request. |
| **Story Points** | 5 |
| **Assignee** | Arin Singh |
| **Priority** | High |
| **Labels/Component** | frontend,backend,db; Leave |
| **Requirement / Use Case** | FR-05; UC-04; SEC-03,04 |
| **Dependencies** | US-01, US-18 |

| | |
|---|---|
| **Title** | US-06 — Review leave and escalate to the Chief Warden |
| **Story** | As a warden, I want to decide assigned leave requests or escalate them, so that requests reach the correct decision-maker. |
| **Description** | Implement Pending, Escalated, Approved and Rejected transitions in a service function. An assigned warden may approve, reject or escalate a Pending request; only the Chief Warden may decide an Escalated request. Save decision-maker, time and remarks for each transition. |
| **Acceptance Criteria** | • An assigned warden can approve, reject or escalate a Pending request; unrelated wardens and students cannot.<br>• Escalated requests appear in the Chief Warden queue and only that role can make their final decision.<br>• Approved and Rejected requests cannot be decided again.<br>• The student sees the current status, remarks and decision time after a valid transition. |
| **Story Points** | 5 |
| **Assignee** | Arin Singh |
| **Priority** | High |
| **Labels/Component** | frontend,backend,workflow; Leave |
| **Requirement / Use Case** | FR-05; UC-04; BR-01 |
| **Dependencies** | US-05 |

## EP-02 — Leave, Attendance and Deviation Records (continued)

| Title | US-07 — Configure sessions and submit location-checked attendance |
|---|---|
| **Story** | As a student, I want to submit attendance during an allowed session, so that the hostel can record my presence using a one-time location check. |
| **Description** | Allow authorized wardens to configure a time window, centre coordinates and permitted radius. Request one browser location sample on submission and validate time and distance in Django. Store the check result and time, not a continuous location history; browser location is a limited signal rather than tamper-proof proof of presence. |
| **Acceptance Criteria** | • Only authorized wardens can configure sessions; the end time must follow the start time and radius must be positive.<br>• A valid submission within the inclusive time window and radius creates one accepted record.<br>• Outside-range, closed-session, denied-permission or unavailable-location attempts do not create accepted attendance.<br>• A database constraint prevents two accepted records for the same student and session, including retries.<br>• No background watcher runs; raw coordinates are not retained in records or application logs after the check. |
| **Story Points** | 8 |
| **Assignee** | Arin Singh |
| **Priority** | High |
| **Labels/Component** | frontend,backend,location; Attendance |
| **Requirement / Use Case** | FR-06; UC-05; NFR-03; SEC-05; BR-05 |
| **Dependencies** | US-01, US-18 |

| Title | US-08 — Record deviations and calculate active weightage |
|---|---|
| **Story** | As a warden, I want to record rule deviations and their weightage, so that students and wardens can see an accurate personal total. |
| **Description** | Link each deviation to a student, an existing rule and the recording warden. Copy the applicable rule weightage into the record so later rule changes do not rewrite historical entries. Sum only active entries and show students a read-only personal history. |
| **Acceptance Criteria** | • Authorized wardens can record a valid student, rule, weightage and remarks; students cannot write records.<br>• The saved entry includes rule ID, recorded-by user and timestamp.<br>• The total equals the sum of active entries; deactivated entries are excluded.<br>• Students cannot read another student's deviations; changing a rule does not alter stored historical weightage. |
| **Story Points** | 5 |
| **Assignee** | Arin Singh |
| **Priority** | Medium |
| **Labels/Component** | frontend,backend,db; Deviations |
| **Requirement / Use Case** | FR-10; UC-09; BR-07 |
| **Dependencies** | US-01, US-03 |

## EP-03 — Study-Room Status and Laundry Tracking

Epic lead: Aryan Singh

Deliver current study-room status and the assigned laundry-day, handover and collection workflow. This epic explicitly excludes room booking, seat reservations and student choice of laundry days.

| | |
|---|---|
| **Title** | US-09 — Display and update study-room Open or Closed status |
| **Story** | As a student, I want to see whether the study room is open, so that I know whether I can use it without making a booking. |
| **Description** | Provide a status card with optional timings and a staff message. The Hostel Director or a specifically permitted staff member may update the record. Keep this a simple status display and do not add rooms, seats or reservation workflows. |
| **Acceptance Criteria** | • Only Open or Closed is accepted as the saved status.<br>• Students see the latest saved status, optional timing/message and update time.<br>• Only permitted staff can update it; the updating user is stored.<br>• No booking button, reservation API or student status-edit action is provided. |
| **Story Points** | 3 |
| **Assignee** | Aryan Singh |
| **Priority** | Medium |
| **Labels/Component** | frontend,backend; Facilities |
| **Requirement / Use Case** | FR-03; UC-02 (legacy; proposed UC-11); BR-02 |
| **Dependencies** | US-01 |

| | |
|---|---|
| **Title** | US-10 — Assign and view the allocated laundry day |
| **Story** | As a student, I want to see the laundry day allocated to me, so that I know when to hand over my clothes. |
| **Description** | Create an allocated-day record linked to each student. Laundry Staff maintains the day while students see a read-only view. Store allocation updates separately from historical handover records so changing a future day does not rewrite past records. |
| **Acceptance Criteria** | • Laundry Staff can assign or update one valid weekday for a student.<br>• The student sees the assigned day or a clear Not assigned message.<br>• A student cannot select, reserve or modify a laundry day through the page or API.<br>• An allocation update preserves earlier handover and collection records. |
| **Story Points** | 3 |
| **Assignee** | Aryan Singh |
| **Priority** | Medium |
| **Labels/Component** | frontend,backend,db; Laundry |
| **Requirement / Use Case** | FR-04; UC-03; BR-03 |
| **Dependencies** | US-01, US-18 |

## EP-03 — Study-Room Status and Laundry Tracking (continued)

| Title | US-11 — Record clothes handed over to laundry |
|---|---|
| **Story** | As a laundry staff member, I want to record clothes received from a student, so that both the student and staff can track the handover. |
| **Description** | Add a received-record form with student, item details and item count. Save the server timestamp and the staff member who accepted the clothes. Keep the student view read-only and avoid inventory or booking features. |
| **Acceptance Criteria** | • A valid student, non-empty item details and positive whole-number count create a Received record.<br>• The record stores the server handover time and receiving staff member.<br>• Missing fields, zero count and negative count are rejected.<br>• Only Laundry Staff can create the record; the student can read only personal records. |
| **Story Points** | 5 |
| **Assignee** | Aryan Singh |
| **Priority** | High |
| **Labels/Component** | frontend,backend,db; Laundry |
| **Requirement / Use Case** | FR-04; UC-03; SEC-03,04 |
| **Dependencies** | US-10 |

| Title | US-12 — Record collection and retain laundry history |
|---|---|
| **Story** | As a laundry staff member, I want to mark received clothes as collected, so that completed laundry records are accurate and traceable. |
| **Description** | Add collection as a transition on an existing received record. Record collection time and the staff member completing it without deleting the handover details. Show the final state and both timestamps in the student history. |
| **Acceptance Criteria** | • Collection is allowed only for an existing Received record.<br>• The record becomes Collected and keeps both handover and collection timestamps.<br>• A repeated collection request does not create another record or overwrite the original completion time.<br>• Students cannot change status or read another student's history. |
| **Story Points** | 3 |
| **Assignee** | Aryan Singh |
| **Priority** | High |
| **Labels/Component** | frontend,backend,workflow; Laundry |
| **Requirement / Use Case** | FR-04; UC-03; BR-04 |
| **Dependencies** | US-11 |

## EP-04 — Complaints, Mess and Hostel Fees

Epic lead: Atiksh Gour

Deliver maintenance follow-up, mess allocation, coupon records and fee-payment status. Payments use provider test mode for development; a simulation may replace that only with faculty approval and must be labelled simulated.

| | |
|---|---|
| **Title** | US-13 — Submit and track a maintenance complaint |
| **Story** | As a student, I want to report a hostel maintenance problem, so that I can follow its progress without repeated visits. |
| **Description** | Create a complaint form using the student's profile room and a selected category. Store description, identifier and initial Open status in Django. Provide a personal complaint list and detail page with status updates. |
| **Acceptance Criteria** | • A valid room, category and non-empty description create an Open complaint with an ID.<br>• Missing or invalid input produces field-level errors and no complaint.<br>• The student can view their own complaint and latest status after refresh.<br>• Another student's complaint cannot be read or modified. |
| **Story Points** | 5 |
| **Assignee** | Atiksh Gour |
| **Priority** | High |
| **Labels/Component** | frontend,backend,db; Complaints |
| **Requirement / Use Case** | FR-07; UC-06; SEC-03,04 |
| **Dependencies** | US-01, US-18 |

| | |
|---|---|
| **Title** | US-14 — Assign complaints and update maintenance progress |
| **Story** | As a maintenance staff member, I want to update assigned complaints, so that students can see the work being done. |
| **Description** | Permit the Hostel Director to assign complaints to Maintenance Staff. Assigned staff may move Open complaints to In Progress and then Resolved with remarks. Preserve status history and display it to the student without granting student edit access. |
| **Acceptance Criteria** | • The Director can assign a complaint to an active Maintenance Staff account.<br>• Only assigned maintenance staff or the Director can update its permitted status transitions.<br>• Every transition stores actor, time and remarks; a resolved complaint cannot silently revert.<br>• The student sees the latest status but cannot edit assignment or staff remarks. |
| **Story Points** | 3 |
| **Assignee** | Atiksh Gour |
| **Priority** | High |
| **Labels/Component** | frontend,backend,workflow; Complaints |
| **Requirement / Use Case** | FR-07; UC-06; SEC-01 |
| **Dependencies** | US-13 |

## EP-04 — Complaints, Mess and Hostel Fees (continued)

| Title | US-15 — Manage mess allocations and active coupons |
|---|---|
| **Story** | As a student, I want to see my assigned mess and valid coupons, so that I know which mess service records apply to me. |
| **Description** | Create simple allocation and coupon records with validity dates and active status. Mess Staff maintains these records through a protected screen. The student page shows current records only; do not add meal ordering, QR redemption or billing. |
| **Acceptance Criteria** | • Only Mess Staff can create or update allocations and coupons.<br>• A student sees the current assigned mess and only active, currently valid coupons.<br>• An invalid validity range or missing coupon code is rejected.<br>• Students cannot edit records or view another student's allocation. |
| **Story Points** | 5 |
| **Assignee** | Atiksh Gour |
| **Priority** | Medium |
| **Labels/Component** | frontend,backend,db; Mess |
| **Requirement / Use Case** | FR-08; UC-07; SEC-03,04 |
| **Dependencies** | US-01, US-18 |

| Title | US-16 — View fees and verify a test payment result |
|---|---|
| **Story** | As a student, I want to pay a listed hostel fee and see its status, so that I can confirm whether the payment was recorded. |
| **Description** | Create a fee-due record and a separate payment-attempt record with fixed-precision amounts. Use provider test mode with server-side result verification and record a unique successful reference; never trust a browser success flag. Keep an attempt Pending until verified and retain an audit history for failure or cancellation. |
| **Acceptance Criteria** | • A student can view personal fees and initiate an attempt for the server-calculated due amount.<br>• Only a verified successful result matching fee, amount and currency marks that fee Paid.<br>• Failed or cancelled attempts leave the fee Unpaid; an unverified result remains Pending.<br>• A unique reference and transaction-safe update prevent duplicate success records.<br>• Passwords, card details and UPI PINs are never collected or stored by HostelSpace; no real charge is made in tests. |
| **Story Points** | 8 |
| **Assignee** | Atiksh Gour |
| **Priority** | High |
| **Labels/Component** | frontend,backend,payments; Fees |
| **Requirement / Use Case** | FR-09; UC-08; NFR-03; SEC-06; BR-06 |
| **Dependencies** | US-01, US-18; provider test access |

## EP-05 — Architecture, Setup and Automated Delivery

Epic lead: Shared

Create the post-SRS architecture, detailed design, reproducible environment and gated delivery process. These are technical enabler stories rather than new hostel features, and ownership is divided among all four members.

| **Title** | US-17 — Baseline architecture and scaffold the monorepo |
|---|---|
| **Story** | As a team developer, I want a shared application skeleton and agreed architecture, so that feature work follows one consistent design. |
| **Description** | Create the frontend and backend skeletons with documented startup steps and a health endpoint. Update the technology and interface descriptions from the earlier SRS to Next.js plus DRF rather than direct Next.js-to-SQL access. Prepare the high-level architecture and agree the shared login, navigation and permission contracts before parallel feature coding. |
| **Acceptance Criteria** | • The repository contains working Next.js and Django/DRF skeletons plus a successful health response.<br>• Architecture and deployment diagrams identify browser, frontend, API, database and external payment/location interfaces.<br>• The document change log records stack updates and the duplicate UC-02 correction proposal without changing FR IDs.<br>• All four members review the architecture and can run the initial skeleton. |
| **Story Points** | 5 |
| **Assignee** | Anvi Srinivas |
| **Priority** | High |
| **Labels/Component** | architecture,frontend,backend; Foundation |
| **Requirement / Use Case** | Post-SRS architecture; FR-01..11 |
| **Dependencies** | None |

| **Title** | US-18 — Agree data models, permissions and API contracts |
|---|---|
| **Story** | As a team developer, I want a reviewed schema and API contract, so that modules integrate without conflicting data definitions. |
| **Description** | Prepare the detailed design with model ownership, fields, keys, constraints and sequence diagrams for critical workflows. Use Django migrations with SQLite convenience locally and PostgreSQL in integration tests and hosted environments. Choose the user model before initial migrations and use settings.AUTH_USER_MODEL in relations. |
| **Acceptance Criteria** | • The design covers profiles, notices/rules, study status, laundry, leave, sessions/attendance, complaints, mess/coupons, fee dues/attempts and deviations.<br>• The endpoint contract defines input, response, role access and validation errors for each FR.<br>• Migrations apply to fresh SQLite and PostgreSQL databases; attendance and payment uniqueness are verified on PostgreSQL.<br>• All four owners approve schema ownership and the role-permission matrix before merging shared models. |
| **Story Points** | 3 |
| **Assignee** | Arin Singh |
| **Priority** | High |
| **Labels/Component** | db,design,backend; Foundation |
| **Requirement / Use Case** | Post-SRS design; FR-01..11; SEC-01..06 |
| **Dependencies** | US-17 |

## EP-05 — Architecture, Setup and Automated Delivery (continued)

| | |
|---|---|
| **Title** | US-19 — Provide a reproducible Docker development environment |
| **Story** | As a team developer, I want a documented container environment and demo data, so that every member can run the same system. |
| **Description** | Add minimal frontend and backend Dockerfiles with a PostgreSQL service in Compose. Keep plain local Next.js plus Django/SQLite as a quick alternative, not as the only integration environment. Provide fictional demo data through Django management commands and document non-destructive reset and migration steps. |
| **Acceptance Criteria** | • A clean clone starts frontend, backend and PostgreSQL using the README steps and Docker Compose.<br>• Database data survives container recreation through a named volume.<br>• Real secrets, SQLite files and generated build folders are excluded from Git and container build contexts.<br>• Each member verifies login to a fictional demo account; demo seeding cannot run in production by default. |
| **Story Points** | 5 |
| **Assignee** | Aryan Singh |
| **Priority** | High |
| **Labels/Component** | docker,db,devops; Foundation |
| **Requirement / Use Case** | Guideline containerization; SEC-02,03 |
| **Dependencies** | US-17, US-18 |

| | |
|---|---|
| **Title** | US-20 — Gate pull requests and deploy reviewed releases |
| **Story** | As a team developer, I want automated quality checks and staged deployment, so that broken changes do not reach the demonstration system. |
| **Description** | Configure GitHub Actions for lint, backend tests, frontend tests, dependency checks and builds. Deploy trusted main-branch commits to staging first, wait for both services and verify the release SHA before production. Store deployment credentials in protected environments and disable any hosting auto-deploy path that bypasses these gates. |
| **Acceptance Criteria** | • Pull requests run required checks without receiving deployment secrets.<br>• A failed required job blocks merging once branch rules are configured.<br>• Only a successful main push can deploy staging; failed staging smoke tests block production.<br>• Production uses a configured approval gate where available and deploys the same reviewed source SHA.<br>• Evidence contains run links, test reports and deployment version; no job reports success merely because a deploy was queued. |
| **Story Points** | 5 |
| **Assignee** | Atiksh Gour |
| **Priority** | High |
| **Labels/Component** | ci-cd,security,devops; Delivery |
| **Requirement / Use Case** | Guideline CI/CD; NFR-05 |
| **Dependencies** | US-17, US-18, US-19 |

## EP-06 — Validation, Maintenance and Final Handoff

Epic lead: Shared

Validate the integrated system against the SRS and preserve genuine review, testing and delivery evidence. Complete the maintenance plan, final report and demonstration without treating documentation or tests as an end-only activity.

| Title | US-21 — Validate usability and assemble the final handoff |
|---|---|
| **Story** | As a faculty evaluator, I want a clear report and role-based demonstration, so that I can verify the completed system and each member's contribution. |
| **Description** | Check forms, empty states and responsive screens across the three SRS browsers. Assemble the report from versioned architecture, design, validation and genuine sprint evidence rather than rewriting it at the end. Coordinate a fifteen-minute rehearsal in which every member explains and demonstrates their own module. |
| **Acceptance Criteria** | • Chrome, Edge and Firefox checks cover labelled forms, field errors and usable phone/desktop layouts.<br>• The final report links all FRs to implemented stories, PRs, tests and validation evidence.<br>• The presentation fits 15 minutes and includes a demonstration by each member.<br>• Outstanding defects and limitations are listed honestly; no unfinished story is labelled implemented. |
| **Story Points** | 3 |
| **Assignee** | Anvi Srinivas |
| **Priority** | Medium |
| **Labels/Component** | documentation,frontend,validation; Handoff |
| **Requirement / Use Case** | NFR-02,04; Final report and demo |
| **Dependencies** | US-01..16; US-22..24 for final sign-off |

| Title | US-22 — Maintain validation cases and verify access rules |
|---|---|
| **Story** | As a project reviewer, I want a testable validation specification and access checks, so that requirements can be confirmed with repeatable evidence. |
| **Description** | Start validation cases while the design is being written and update actual results after each increment. Exercise role boundaries, record ownership, CSRF handling, invalid inputs and one-time location use only against the team's local or staging system. Keep unit-test coverage reports and explicit manual checks for behavior not demonstrated by automation. |
| **Acceptance Criteria** | • Every FR, NFR and SEC ID has a validation case with preconditions, steps and expected result.<br>• Protected student records reject access by another student and by unrelated staff roles in seeded tests.<br>• The attendance boundary and retry cases pass on PostgreSQL without retaining raw location history.<br>• Failed cases create linked Jira bugs and are retested; actual-result and evidence fields are filled only after execution. |
| **Story Points** | 3 |
| **Assignee** | Arin Singh |
| **Priority** | High |
| **Labels/Component** | testing,security,validation; Validation |
| **Requirement / Use Case** | FR-01..11; NFR-01..05; SEC-01..06 |
| **Dependencies** | US-18; US-01..16 for final validation |

## EP-06 — Validation, Maintenance and Final Handoff (continued)

| Title | US-23 — Run integrated workflows and reliability checks |
|---|---|
| **Story** | As a team tester, I want repeatable end-to-end and reliability checks, so that the combined frontend and API work as one system. |
| **Description** | Create a small Playwright suite for login, leave decisions, laundry transitions and complaint progress using fictional users. Run PostgreSQL integration tests for uniqueness and perform the agreed warm-environment performance check. Record repeatable steps for mess, payments and deviations where manual validation is clearer than additional automation. |
| **Acceptance Criteria** | • Critical end-to-end flows pass against the integrated Next.js and Django staging release.<br>• Attendance retries and payment-success retries do not create duplicate records in PostgreSQL tests.<br>• The proposed NFR-01 check uses 20 signed-in simulated users and 100 warm-page visits, measuring navigation-to-usable-page time with p95 at most 3 seconds; obtain faculty/team approval first.<br>• The report states browser, environment, dataset, run time and observed failures; a failed target is not reported as passed. |
| **Story Points** | 8 |
| **Assignee** | Aryan Singh |
| **Priority** | High |
| **Labels/Component** | testing,integration,performance; Validation |
| **Requirement / Use Case** | NFR-01,03,04,05; FR-01..11 |
| **Dependencies** | US-19, US-20; US-01..16 for final regression |

| Title | US-24 — Verify release recovery and complete maintenance guidance |
|---|---|
| **Story** | As a release maintainer, I want a tested backup, rollback and payment-validation runbook, so that the demonstration system can be recovered and maintained. |
| **Description** | Test backup and restore with synthetic PostgreSQL data and record a known-good application release. Recheck fee success, failure, pending and duplicate-result cases in test mode before release. Document ownership of dependency updates, bug triage and release recovery without storing credentials or database dumps in the repository. |
| **Acceptance Criteria** | • A test backup restores successfully into a separate database and expected record counts match.<br>• A staging rollback identifies and restores a known-good frontend/backend release; schema compatibility is checked first.<br>• Payment acceptance cases pass using provider test mode and no live funds or private student data.<br>• The maintenance plan names owners, weekly dependency review, backup steps and incident/bug reporting steps. |
| **Story Points** | 3 |
| **Assignee** | Atiksh Gour |
| **Priority** | High |
| **Labels/Component** | maintenance,ci-cd,testing; Handoff |
| **Requirement / Use Case** | FR-09; NFR-03; SEC-06; Maintenance |
| **Dependencies** | US-16, US-20; US-23 before release |

## 1. Jira Board Plan — traceability and setup finish

| SRS requirement | Stories | Functional owner |
|---|---|---|
| FR-01 / UC-01 | US-01, US-02 | Anvi |
| FR-02 / UC-02 | US-03 | Anvi |
| FR-03 / legacy UC-02; proposed UC-11 | US-09 | Aryan |
| FR-04 / UC-03 | US-10, US-11, US-12 | Aryan |
| FR-05 / UC-04 | US-05, US-06 | Arin |
| FR-06 / UC-05 | US-07 | Arin |
| FR-07 / UC-06 | US-13, US-14 | Atiksh |
| FR-08 / UC-07 | US-15 | Atiksh |
| FR-09 / UC-08 | US-16 | Atiksh |
| FR-10 / UC-09 | US-08 | Arin |
| FR-11 / UC-10 | US-04 | Anvi |
| NFR-01..05 / SEC-01..06 | Feature criteria + US-20, US-21, US-22, US-23, US-24 | All owners; Arin coordinates validation |

Record an actual trace chain as work happens: FR-05 → US-05 → actual Jira key → design section → leave module/PR → TC-FR05-01 → executed validation result. The earlier D-01/C-01/TC-01 values were planned references, not completed evidence. Replace them with real references rather than claiming all links already exist.

### Create these first, before starting features

• Set up the six epics and copy all stories; leave unfinished issues in To Do.

• Put US-17–US-20 in Sprint 1, with US-01 and the early case-design part of US-22.

• Verify assignees and dependency links; attach the current SRS and record the proposed corrections.

• Agree the review rotation, Definition of Done and target dates with all four members.

# 2. Tech Stack & Architecture Explanation

Use a small two-application monorepo: Next.js for the interface and Django/DRF for all business logic. This is compatible and lets each feature owner work end to end, but it is more setup than Django templates alone. Keep one API, one database and simple client forms; do not add microservices, Redis, Celery, Kubernetes, Jenkins or a second backend.

| Layer / tool | What it does and why selected |
|---|---|
| Next.js + JavaScript | Routes and renders pages; shared React components handle forms, status cards and tables. Use the App Router with JavaScript to match the existing frontend scope. |
| Tailwind CSS | One styling approach for spacing, forms and responsive layouts. Use simple utility styles and a few shared components; do not also introduce Bootstrap. |
| Python + Django + DRF | Django owns authentication, permissions, validation, models and migrations. DRF serializers/views expose JSON APIs for Next.js; do not use Django templates for the same screens. |
| PostgreSQL / SQLite | PostgreSQL for CI integration, staging and production. SQLite is optional for quick local work; Django selects it only in local development when DATABASE_URL is absent. |
| Git + GitHub | History, branches, peer reviews and individual contribution evidence. Commit lockfiles, migrations and documentation, not generated outputs or secrets. |
| GitHub Actions | Runs quality gates on PRs and trusted pushes. It is a delivery tool, not part of each student request, and replaces the need for Jenkins. |
| pytest + pytest-django + pytest-cov | Backend model, permission, service and API tests with coverage. Use PostgreSQL for concurrency/constraint behavior that SQLite cannot prove. |
| Jest + React Testing Library | Tests synchronous/client components and form behavior. Use a small Playwright suite for complete flows; Jest is not sufficient for async Server Components [R1]. |
| Ruff / ESLint / security checks | Ruff for Python; ESLint for JavaScript; Bandit, pip-audit and npm audit for code/dependency findings. Findings need triage; scanners are not proof of security. |
| Docker + Compose | Small reproducible integration setup: frontend, API and PostgreSQL. Daily local Docker use may be optional, but build/run evidence remains in the course plan. |
| Vercel + Render | Vercel hosts Next.js; Render hosts Django and managed PostgreSQL. Use separate staging/production resources and check current plan, budget and team-collaboration restrictions before committing. |

Suggested baseline: Python 3.12, supported Django 5.2 LTS patch, a compatible DRF release, Node.js 22 LTS and PostgreSQL 16. At setup time verify support, select a supported Next.js release, pin Python dependencies and commit package-lock.json; do not use floating “latest” in deployment.

## 2. Architecture — request, identity and data flow

```
Student / Staff browser
  | HTTPS, page request or form interaction
  v
Next.js pages and shared React components
  | same-origin /api/* proxy, JSON + session + CSRF
  v
Django REST Framework
  | authenticate -> role/object permission -> validate
  v
Django service functions and ORM
  | parameterized queries / transactions
  v
PostgreSQL (staging and production)

GitHub Actions is separate:
PR / trusted main push -> tests/builds -> staged releases
```

For a leave request, Next.js renders the form and sends the dates and reason to /api/leave/. The same-origin proxy forwards it to Django, which checks the active session, student ownership and date rules before saving through the ORM. PostgreSQL stores the record and Django returns its ID and Pending status as JSON; Next.js then updates the page. GitHub Actions does not process this student request: it checks code changes on PRs and starts the gated deployment sequence after a reviewed merge to main.

## Authentication: keep one source of truth

• Use Django sessions, not separate NextAuth users or homemade JWT storage. Route browser /api/* calls through a Next.js rewrite/proxy to the trusted Django origin. Session cookies must be host-only, HttpOnly and Secure in deployment; ensure proxy responses preserve Set-Cookie.

• Provide a CSRF bootstrap endpoint and send the token header on writes. Protect login explicitly; anonymous login must not rely solely on DRF SessionAuthentication's authenticated-request CSRF check. Keep Django CSRF middleware enabled and trust only the exact frontend origins.

• Test cookie forwarding, CSRF, logout and redirects on the hosted environment in week one. Disable caching of authenticated API responses and never put sessions, passwords or provider secrets in localStorage or NEXT_PUBLIC variables.

• Django is authoritative even if navigation hides buttons. All services enforce active role and record ownership. Django superuser access is for controlled maintenance, not a substitute for the Hostel Director dashboard.

## Database and integration guardrails

SQLite has different locking, typing and constraint behavior from PostgreSQL. Commit portable Django migrations; run all migrations and critical retry/concurrency tests on PostgreSQL before merge. Use Decimal amounts and database uniqueness for payment references and attendance; never use a local SQLite file on ephemeral hosted storage. Production must fail configuration if DATABASE_URL is missing rather than silently falling back to SQLite.

Browser location is requested once and is not a tamper-proof attendance guarantee. Test denied/low-accuracy readings and document the agreed accuracy policy. Use a payment provider's test environment with verified server results; choosing the provider and obtaining credentials is an early dependency, not something already completed.

# 3. CI/CD Pipeline Explanation

CI means Continuous Integration: automatically check each proposed change. CD here means Continuous Delivery when production waits for approval, or Continuous Deployment when it proceeds without that manual step. Production is not deployed from a feature branch or untrusted PR.

| Step | Execution and gate |
|---|---|
| 1. Push / PR | Feature branch → PR to main. Run CI on every PR update and on the merged main commit. |
| 2. Lint and security | Ruff, ESLint, Bandit and dependency audits. Fix blocking findings; any exception must be documented, reviewed and time-limited. |
| 3. Backend tests | Run Django checks and pytest with a PostgreSQL service; check pending model changes and collect coverage. |
| 4. Frontend tests | Run Jest/React Testing Library with coverage after backend tests pass; an empty test suite is not a pass. |
| 5. Build | Build Next.js and both container images. This checks packaging, not whether every feature works. |
| 6. Deploy staging | Only on a trusted main push. Deploy frontend and backend for the same commit; wait for readiness and check the reported release SHA. |
| 7. Staging smoke gate | Run login/API health and critical synthetic-user flows. A queued deployment is not a successful deployment. |
| 8. Approval → production | Where supported, require a production environment reviewer other than the author. Deploy the same source SHA, then run smoke checks and record the release. |
| 9. Failure / recovery | Stop the sequence on failure. Keep the last known-good release and use the maintenance runbook; application rollback does not automatically reverse database migrations. |

## Branch and deployment protection

Protect main: no direct pushes or force pushes, one independent approving review, and required lint, backend_tests, frontend_tests and build checks. Re-run checks after updates and restrict bypass permission. GitHub does not block failed code merely because Actions exists; these rules must be configured. Environment reviewers and protection features depend on repository visibility and plan [R2]. If the approval feature is unavailable, use an explicitly reviewed manual production release of the tested SHA and do not claim an automatic approval gate.

Use Actions as the single release orchestrator. Disable independent Vercel/Render commit-based production deployment that would race past staging. Render’s After CI Checks Pass option is useful for simpler flows but is not by itself the staging-smoke/production-approval chain in this plan [R3].

## 3. Sample GitHub Actions YAML — checks

Blueprint for .github/workflows/ci.yml, not an already configured or production-ready workflow. US-20 must implement the deployment scripts, project scripts, settings, secrets and reports described below. Action versions shown are examples; verify supported versions and pin reviewed commit SHAs before use.

```
name: HostelSpace CI and Delivery
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
permissions:
  contents: read
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: false
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - uses: actions/setup-node@v4
        with: {node-version: '22'}
      - run: pip install -r backend/requirements-dev.txt
      - run: npm ci --prefix frontend
      - run: ruff check backend
      - run: bandit -r backend/apps -ll -x tests
      - run: pip-audit -r backend/requirements.txt
      - run: npm run lint --prefix frontend
      - run: npm audit --prefix frontend --audit-level=high
```

## 3. Sample YAML — tests and builds (continued)

```
backend_tests:
  needs: lint
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:16
      env:
        POSTGRES_DB: hostelspace_ci
        POSTGRES_USER: ci
        POSTGRES_PASSWORD: ci_test_only
      ports: ['5432:5432']
      options: >-
        --health-cmd pg_isready --health-interval 10s
        --health-timeout 5s --health-retries 5
  env:
    DATABASE_URL: postgresql://ci:ci_test_only@localhost:5432/hostelspace_ci
    DJANGO_SETTINGS_MODULE: config.settings.ci
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: {python-version: '3.12'}
    - run: pip install -r backend/requirements-dev.txt
    - run: |
        cd backend
        python manage.py check
        python manage.py makemigrations --check --dry-run
        python manage.py migrate --noinput
        pytest --cov=apps --cov-report=xml --cov-fail-under=80

frontend_tests:
  needs: backend_tests
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: {node-version: '22'}
    - run: npm ci --prefix frontend
    - run: npm run test:ci --prefix frontend

build:
  needs: frontend_tests
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: {node-version: '22'}
    - run: npm ci --prefix frontend
    - run: npm run build --prefix frontend
    - run: docker build -t hostelspace-api ./backend
    - run: docker build -t hostelspace-web ./frontend
```

## 3. Sample YAML — gated deployment (continued)

```
staging:
  needs: build
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment: staging
  env:
    RENDER_API_KEY: ${{ secrets.RENDER_API_KEY }}
    RENDER_SERVICE_ID: ${{ vars.RENDER_SERVICE_ID }}
    VERCEL_TOKEN: ${{ secrets.VERCEL_TOKEN }}
    VERCEL_ORG_ID: ${{ vars.VERCEL_ORG_ID }}
    VERCEL_PROJECT_ID: ${{ vars.VERCEL_PROJECT_ID }}
    FRONTEND_URL: ${{ vars.FRONTEND_URL }}
    BACKEND_URL: ${{ vars.BACKEND_URL }}
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: {node-version: '22'}
    - run: bash scripts/deploy.sh staging "$GITHUB_SHA"
    - run: bash scripts/smoke.sh staging "$GITHUB_SHA"

production:
  needs: staging
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment: production  # Configure reviewer in repository settings.
  env:
    RENDER_API_KEY: ${{ secrets.RENDER_API_KEY }}
    RENDER_SERVICE_ID: ${{ vars.RENDER_SERVICE_ID }}
    VERCEL_TOKEN: ${{ secrets.VERCEL_TOKEN }}
    VERCEL_ORG_ID: ${{ vars.VERCEL_ORG_ID }}
    VERCEL_PROJECT_ID: ${{ vars.VERCEL_PROJECT_ID }}
    FRONTEND_URL: ${{ vars.FRONTEND_URL }}
    BACKEND_URL: ${{ vars.BACKEND_URL }}
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: {node-version: '22'}
    - run: bash scripts/deploy.sh production "$GITHUB_SHA"
    - run: bash scripts/smoke.sh production "$GITHUB_SHA"
```

## Required setup behind the YAML

Set backend/requirements-dev.txt to include runtime dependencies, pytest-django, pytest-cov, Ruff, Bandit and pip-audit. config.settings.ci must provide a test-only secret, parse DATABASE_URL and never load production credentials. Configure Jest's test:ci command and an agreed coverage threshold; 80% is a proposed team target, not a faculty-mandated percentage. Store coverage/JUnit outputs as Actions artifacts even when tests fail.

## 3. Deployment implementation contract

| File / setting | What US-20 must implement |
|---|---|
| scripts/deploy.sh | Accept environment and source SHA. Install/use a pinned provider CLI, deploy the exact backend commit and frontend source, apply reviewed migrations, wait for both deployments and fail on timeout or provider failure. Never treat an asynchronous deploy-hook response as completion. |
| scripts/smoke.sh | Check both version endpoints report the expected SHA, then run health and fictional-user login/critical-flow tests. Return non-zero on mismatch or failure. Keep synthetic test credentials in environment secrets, not YAML. |
| Two GitHub environments | Create staging and production with different service/project IDs and secrets. Add a production required reviewer where the GitHub plan supports it. Only trusted main runs can use deployment secrets. |
| Hosting configuration | Vercel frontend project root = frontend; Render API root = backend. Use separate PostgreSQL databases for staging/production, disable DEBUG and enforce HTTPS. Never connect staging tests to production data. |
| Immutable source and config | Pass the actual SHA to providers and expose it as release metadata. A Next.js rebuild for environment-specific API configuration is acceptable if it uses the same locked source and dependencies; do not claim the binaries are identical. |
| Evidence and audits | Upload test/coverage artifacts; record scanner findings and review decisions. Keep secrets out of output, URLs, screenshots and Git. No pull_request_target workflow should execute untrusted PR code with deployment credentials. |
| Local readiness and health | Builds should not require a live production database or student dataset. Health responses expose readiness/version only. Use Gunicorn or the platform-supported production server, not Django runserver. |
| Rollback / budget | Record a tested known-good frontend/API pair. Use backward-compatible migrations; database restore needs a separate reviewed procedure. Confirm current hosting and reviewer-feature costs, quotas and team permissions before setup. |

Vercel documents a GitHub Actions/CLI deployment pattern [R4]. The scripts above are deliberate integration boundaries in this high-level example, not hidden completed work. Implement and test them during US-20; deployment remains unconfigured until that story passes its acceptance criteria.

# 4. Folder Structure (mapped to who owns what)

One repository, hostelspace. A feature owner owns both its frontend and backend folders, not only one technology layer. The shared accounts model, API client, settings and schema contracts require review by affected owners before changes.

```
hostelspace/
├── frontend/                          # Anvi coordinates
│   ├── src/
│   │   ├── app/
│   │   │   ├── (auth)/login/          # Anvi — US-01
│   │   │   ├── (portal)/
│   │   │   │   ├── profile/           # Anvi — US-02
│   │   │   │   ├── information/       # Anvi — US-03
│   │   │   │   ├── director/          # Anvi — US-04
│   │   │   │   ├── leave/             # Arin — US-05/06
│   │   │   │   ├── attendance/        # Arin — US-07
│   │   │   │   ├── deviations/        # Arin — US-08
│   │   │   │   ├── study-room/        # Aryan — US-09
│   │   │   │   ├── laundry/           # Aryan — US-10/11/12
│   │   │   │   ├── complaints/        # Atiksh — US-13/14
│   │   │   │   ├── mess/              # Atiksh — US-15
│   │   │   │   └── fees/              # Atiksh — US-16
│   │   │   ├── layout.js
│   │   │   └── page.js
│   │   ├── components/ui/             # Anvi; reviewed shared UI
│   │   ├── features/                  # matching module owners
│   │   └── lib/api.js                 # Anvi; API/session client
│   ├── public/                        # Anvi; approved assets
│   ├── next.config.mjs                # Anvi + Atiksh; API proxy
│   ├── jest.config.mjs
│   ├── eslint.config.mjs
│   ├── package.json
│   ├── package-lock.json
│   ├── Dockerfile                     # Aryan
│   └── .env.example
├── backend/
│   ├── config/                        # Arin + Anvi; shared
│   │   ├── settings/
│   │   │   ├── base.py
│   │   │   ├── local.py
│   │   │   ├── ci.py
│   │   │   └── production.py
│   │   ├── urls.py
│   │   └── wsgi.py
│   ├── apps/
│   │   ├── accounts/                  # Anvi — US-01/02
│   │   │   └── management/commands/   # Aryan; demo data only
│   │   ├── information/               # Anvi — US-03
│   │   ├── director_dashboard/        # Anvi — US-04
│   │   ├── leave_attendance/          # Arin — US-05..08
│   │   ├── facilities/                # Aryan — US-09..12
│   │   └── student_services/          # Atiksh — US-13..16
│   ├── manage.py
│   ├── pyproject.toml                 # Arin; pytest/Ruff config
│   ├── requirements.txt
│   ├── requirements-dev.txt
│   ├── Dockerfile                     # Aryan
│   └── .env.example
```

## 4. Repository tree — supporting folders

```
├── .github/                          # Atiksh
│   ├── workflows/ci.yml
│   ├── pull_request_template.md      # Anvi
│   └── CODEOWNERS                    # real GitHub handles only
├── tests/
│   ├── e2e/                          # Aryan — US-23
│   ├── system/                       # Aryan + Anvi — US-21/23
│   └── security/                     # Arin — US-22
├── scripts/                          # Atiksh — US-20/24
│   ├── deploy.sh
│   └── smoke.sh
├── docs/
│   ├── 01-synopsis/                  # Anvi; existing + revisions
│   ├── 02-srs/                       # Anvi; all owners review
│   ├── 03-architecture/              # Anvi — US-17
│   ├── 04-design/                    # Arin — US-18; all modules
│   ├── 05-validation/                # Arin — US-22; live RTM
│   ├── 06-testing/                   # Aryan — US-23
│   ├── 07-maintenance/               # Atiksh — US-24
│   ├── 08-final-report/              # Anvi — US-21; team input
│   ├── 09-presentation/              # Anvi — US-21; all speak
│   ├── decisions/                   # Anvi coordinates
│   └── prompts/                     # each member's truthful log
├── evidence/                         # Aryan curates; all supply
│   ├── README.md                     # links to genuine runs/PRs
│   └── sanitized-screenshots/
├── compose.yaml                      # Aryan — US-19
├── .gitignore                        # Aryan; team-reviewed
├── .dockerignore                     # plus per-build-context ignores
├── README.md                         # Anvi coordinates
└── CONTRIBUTING.md                   # Anvi; team agreement

Each Django app has:
  __init__.py, apps.py, models.py, serializers.py, views.py,
  urls.py, permissions.py, services.py, admin.py,
  migrations/__init__.py, tests/__init__.py, test_*.py

Each frontend feature keeps its components and *.test.jsx files
in src/features/<module>/; API calls use the shared lib/api.js.
```

Do not create empty services or permissions files merely to match the tree; add them when needed. Include Python package __init__.py files and set each AppConfig to its actual apps.<module> path. Do not keep both tests.py and a tests/ package in the same Django app.

## 4. Folder-to-owner map and repository hygiene

| Folder / module | Accountable owner | Story / review notes |
|---|---|---|
| frontend: login, profile, information, director; shared UI/API client | Anvi | US-01..04, US-17; Arin reviews auth and schema changes. |
| backend/apps/accounts, information, director_dashboard | Anvi | End-to-end ownership; Director queries use other modules without rewriting their models. |
| frontend: leave, attendance, deviations; backend/apps/leave_attendance | Arin | US-05..08; owns related unit/API/component tests. |
| frontend: study-room, laundry; backend/apps/facilities | Aryan | US-09..12; owns allocation and received/collected rules. |
| frontend: complaints, mess, fees; backend/apps/student_services | Atiksh | US-13..16; payment verification stays server-side. |
| backend/config, schema/API design | Arin; Anvi for auth | US-18. Each module owner writes its migrations; coordinate shared-user changes before merging. |
| Dockerfiles, compose.yaml, demo commands | Aryan | US-19. Django and Next.js containers use the same reviewed config contracts. |
| .github/workflows, scripts, hosting settings | Atiksh | US-20, US-24; require an independent review of release changes. |
| tests/e2e, tests/system, evidence | Aryan; Anvi for usability | US-23, US-21. Owners still write their own feature tests. |
| tests/security, docs/05-validation | Arin | US-22; all members execute and record their assigned requirements. |
| docs/07-maintenance | Atiksh | US-24; backup, dependency updates and rollback. |
| docs/01..03, decisions, final report, presentation | Anvi | US-17, US-21; each member contributes module detail and demonstration. |
| docs/prompts/<member>/ | Each named member | Save useful prompts, resulting changes, review decisions and test results; no secrets or private student data. |

## Keep the repository small and safe

Commit source, lockfiles, migrations, reviewed diagrams and small sanitized evidence. Ignore .env, .venv, __pycache__, db.sqlite3, .next, node_modules, collected static files, coverage outputs, logs and database dumps; use Actions artifacts for generated reports. Commit only .env.example with non-secret sample values. A root .dockerignore does not cover builds rooted in frontend/ or backend/: put an ignore file in each actual build context.

Settings: server-only DJANGO_SECRET_KEY, DATABASE_URL, payment keys and deployment tokens; NEXT_PUBLIC values are visible to browsers and must never hold secrets. Local settings may allow localhost; production must explicitly set allowed hosts, trusted CSRF origins, HTTPS cookies and DEBUG=False.

# 5. GitHub Workflow / Team Account Usage Plan

Create one team-owned GitHub repository or a repository with all four members added as collaborators. Every member uses their own account, two-factor authentication and Git author identity. Never share a password, impersonate another member or rewrite authorship to make contribution totals look balanced.

## Branches, commits and reviews

Use protected main plus short-lived feature branches; omit develop to keep this small-team workflow simple. Reviewed main is integration-ready, staging is deployed from it and production promotion happens only after gates. Never keep six-week personal branches that merge only at the end.

| Member | Branch example | Primary reviewer |
|---|---|---|
| Anvi | feature/anvi-US-01-login | Arin |
| Arin | feature/arin-US-05-leave | Aryan |
| Aryan | feature/aryan-US-11-handover | Atiksh |
| Atiksh | feature/atiksh-US-16-fees | Anvi |

After Jira creation, include the actual HOST key in the branch/PR instead of or alongside US IDs. Do not invent HOST-21 as if Jira issued it. Use fix/ for bug branches and chore/ for setup. Review payments, auth and deployment with a second affected owner when possible; the author cannot approve their own PR.

```
feat(leave): validate request dates [US-05]
fix(laundry): reject collection without handover [US-12]
test(fees): cover repeated verified payment result [US-16]
chore(ci): add PostgreSQL test service [US-20]
docs(design): record shared user model decision [US-18]
```

Each PR states the story/actual Jira key, changed paths, acceptance criteria covered, migrations, tests run, screenshots if useful, and known gaps. Review the diff and test evidence, not just the green badge. If squashing, retain accurate author/co-author attribution and preserve the PR discussion for evidence.

## Realistic daily cadence

• On a working day: pull main, implement one coherent change, run affected checks, then commit and push by the end of the work session. A useful commit may be a schema with its tests, an endpoint, a form or a correction; it should not knowingly break the branch.

• Open a draft PR once a small vertical slice exists, normally within 1–2 active workdays. Request review only when it is runnable and explainable. Aim to review teammates within one working day, subject to class availability.

• Do not fabricate daily activity or split changes into meaningless commits. These dates are planned work sessions, not evidence of completed work. Tests are added with behavior, and later regression sessions extend them rather than being the first time tests are written.

## 5. Dated commit/push plan — setup to working features

All dates are proposed local working dates in Asia/Kolkata, 2026. Push at the end of each actual session, not at a fixed artificial hour. Team availability may move a session; preserve dependency order.

| Owner | Planned meaningful commit sequence |
|---|---|
| Anvi | 7 Sep: monorepo skeleton and README (US-17). 9 Sep: architecture/API session decision. 11 Sep: login/logout slice with tests (US-01). 15 Sep: profile API/form (US-02). 17 Sep: notice/rule UI and permissions (US-03). 22 Sep: Director role/account endpoints (US-04). 24 Sep: summary queries and empty states. |
| Arin | 7 Sep: reviewed user/schema decisions (US-18). 10 Sep: portable models/API contract. 14 Sep: leave model/API tests (US-05). 16 Sep: leave form and errors. 18 Sep: approval/escalation service tests (US-06). 21 Sep: attendance-session model/configuration (US-07). 23 Sep: location UI and duplicate checks. 25 Sep: deviation totals and rule links (US-08). |
| Aryan | 8 Sep: Dockerfiles and ignore rules (US-19). 10 Sep: Compose/database health and fictional seed data. 15 Sep: study-room status API/card (US-09). 17 Sep: assigned-day model/view (US-10). 22 Sep: handover form/service/tests (US-11). 24 Sep: collection transition and history (US-12). |
| Atiksh | 8 Sep: initial lint/test Actions jobs (US-20). 11 Sep: gated staging skeleton and version checks. 14 Sep: complaint model/API/tests (US-13). 16 Sep: complaint form/list. 18 Sep: staff transition/assignment checks (US-14). 21 Sep: mess allocation/coupon tests (US-15). 23 Sep: fee-due/payment-attempt model (US-16). 25 Sep: verified test-payment result handling. |

## Planned small releases, not one final dump

11 September: environment/auth checkpoint. 18 September: first integrated student/staff workflows. 25 September: feature-complete candidate, subject to tests. Each checkpoint is a reviewed main state with a staging run; it does not imply production is ready. If a story misses its criteria, leave it incomplete and reschedule it.

## 5. Dated commit/push plan — integration to handoff

| Owner | Planned correction, test and release commits |
|---|---|
| Anvi | 29 Sep: cross-role UI/error-state fixes. 1 Oct: browser/responsive evidence (US-21). 6 Oct: final-report references and diagrams. 8 Oct: presentation draft and own module script. 13 Oct: rehearsal corrections. 15 Oct: accepted documentation release. |
| Arin | 28 Sep: auth/ownership/CSRF validation cases (US-22). 30 Sep: attendance time/radius/retry boundary tests. 2 Oct: validated model/RTM corrections. 7 Oct: regression fixes and trace links. 9 Oct: validation actual results. 14 Oct: final retest and own demo notes. |
| Aryan | 28 Sep: critical-flow Playwright suite (US-23). 30 Sep: Docker clean-clone and PostgreSQL retry tests. 2 Oct: performance/environment results. 5 Oct: integration bug fixes. 8 Oct: browser regression evidence. 13 Oct: demo rehearsal fixes. 15 Oct: final clean-environment run. |
| Atiksh | 29 Sep: payment failed/pending/duplicate cases. 1 Oct: staging deploy-readiness and version-gate fixes. 6 Oct: backup/restore and rollback evidence (US-24). 8 Oct: maintenance runbook and dependency review. 14 Oct: release candidate and test-payment retest. 16 Oct: reviewed release tag and deployment evidence. |

## Shared-file rules

• Anvi coordinates the shared frontend layout and session client; Arin coordinates user/schema contracts; Aryan coordinates container changes; Atiksh coordinates workflows and hosting. Feature ownership never gives permission to overwrite another module without review.

• Each person creates migrations only for their own model changes, rebases on main and checks for migration conflicts. Decide a custom user model before the first committed database baseline.

• CODEOWNERS must contain actual GitHub usernames gathered during setup. This plan names people rather than inventing handles. Give deployment secrets only to the release workflow and authorized maintainers.

• Create a release tag such as v1.0.0 only when its checks pass. Keep the tag, deployed source SHA, Jira release notes and test evidence consistent.

# 6. Prompts to Give Antigravity (per module)

Use the following as six copy-ready prompt packs. For the four functional packs, the named owner should request one story at a time in dependency order. For shared packs, the story-to-person mapping is explicit; each person selects only their assigned story. Each pack includes the actual story IDs/titles, paths and acceptance requirements instead of asking the tool to invent scope.

## Common instructions to paste before every pack

```
Work on HostelSpace, Team 11. Read the repository README, current SRS,
architecture decisions and API contract first. Use Next.js/JavaScript
with Tailwind for the UI, Django REST Framework for the API, Django
sessions/CSRF for auth, PostgreSQL for CI/staging/production and SQLite
only for quick local development. Do not add a second auth system.
Implement only the selected story below. Preserve unrelated modules.
Do not run a migration against production, publish a deployment, use
real payments, delete data, commit, or push without explicit approval.
Before editing, list the affected files and any missing decisions.
Implement server-side permissions and validation, migrations if needed,
pytest backend tests and Jest/React Testing Library UI tests.
Use synthetic data. Never print secrets or put them in code or prompts.
After changes, list commands actually run, real results, changed files,
acceptance criteria covered and remaining gaps. Do not claim tests passed
unless you ran them. Explain the implementation so I can demonstrate it.
Return the diff for human review; do not weaken tests to make CI green.
```

Keep the meaningful prompts, decisions and reviewed output in docs/prompts/<member>/ and link them to the story. AI assistance does not transfer ownership or replace code understanding. The course evaluation may ask for prompts and explanations; keep a truthful record.

## 6. EP-01 prompt — Student Access and Hostel Information

Assigned owner: Anvi Srinivas. Paste this after the common instructions.

Implement EP-01 — Student Access and Hostel Information for HostelSpace. Work one story at a time in dependency order, not the entire epic in one unreviewable change.

### US-01 — Sign in, sign out and open the correct dashboard | Owner: Anvi

Paths: frontend/src/app/(auth)/login/; frontend/src/lib/api.js; backend/apps/accounts/.

Satisfy: 1) An active account with valid credentials reaches its permitted dashboard; invalid or inactive accounts are denied. 2) Protected APIs reject missing sessions and disallowed roles even if a URL is entered directly. 3) Logout invalidates the session; a subsequent protected request is denied. 4) Session cookies are HttpOnly and Secure in HTTPS deployment; missing or invalid CSRF tokens are rejected on writes.

### US-02 — View and update my student profile | Owner: Anvi

Paths: frontend/src/app/(portal)/profile/; backend/apps/accounts/.

Satisfy: 1) A student sees only their own profile; another student record cannot be read or changed. 2) A valid phone update persists after a refresh. 3) Empty or malformed phone input is rejected with a field message. 4) Attempts to change SRN, room, role or another protected field are rejected or excluded from the permitted update.

### US-03 — Publish and read hostel notices, contacts and rules | Owner: Anvi

Paths: frontend/src/app/(portal)/information/; backend/apps/information/.

Satisfy: 1) Students see active notices whose active-date window includes the current time. 2) Authorized content staff can create, edit and deactivate information; students cannot. 3) Missing notice titles and invalid active-date ranges return validation errors. 4) Rules and hostel contacts remain readable; a referenced rule is deactivated rather than deleted.

### US-04 — Manage authorized users and view Director summaries | Owner: Anvi

Paths: frontend/src/app/(portal)/director/; backend/apps/director_dashboard/.

Satisfy: 1) Non-Director users cannot access the Director management endpoints. 2) The Director can activate or deactivate accounts and assign permitted roles; deactivated accounts lose access. 3) Only one active Chief Warden is allowed by the agreed role-management rule. 4) Summary counts match seeded source records; empty datasets show zero rather than an error.

Use the implementation descriptions and SRS mappings in Section 1. Return a small reviewed slice with tests for these conditions; identify unmet dependencies rather than inventing records, API credentials or passing evidence.

## 6. EP-02 prompt — Leave, Attendance and Deviation Records

Assigned owner: Arin Singh. Paste this after the common instructions.

Implement EP-02 — Leave, Attendance and Deviation Records for HostelSpace. Work one story at a time in dependency order, not the entire epic in one unreviewable change.

### US-05 — Submit and track a leave request | Owner: Arin

Paths: frontend/src/app/(portal)/leave/; backend/apps/leave_attendance/.

Satisfy: 1) Valid dates and a non-empty reason create one request with an ID and Pending status. 2) An end date before the start date or missing required input is rejected. 3) The request stores the assigned warden and is visible to that warden. 4) Students can read their own history but cannot read another student's request.

### US-06 — Review leave and escalate to the Chief Warden | Owner: Arin

Paths: frontend/src/app/(portal)/leave/review/; backend/apps/leave_attendance/.

Satisfy: 1) An assigned warden can approve, reject or escalate a Pending request; unrelated wardens and students cannot. 2) Escalated requests appear in the Chief Warden queue and only that role can make their final decision. 3) Approved and Rejected requests cannot be decided again. 4) The student sees the current status, remarks and decision time after a valid transition.

### US-07 — Configure sessions and submit location-checked attendance | Owner: Arin

Paths: frontend/src/app/(portal)/attendance/; frontend/src/features/attendance/; backend/apps/leave_attendance/.

Satisfy: 1) Only authorized wardens can configure sessions; the end time must follow the start time and radius must be positive. 2) A valid submission within the inclusive time window and radius creates one accepted record. 3) Outside-range, closed-session, denied-permission or unavailable-location attempts do not create accepted attendance. 4) A database constraint prevents two accepted records for the same student and session, including retries. 5) No background watcher runs; raw coordinates are not retained in records or application logs after the check.

### US-08 — Record deviations and calculate active weightage | Owner: Arin

Paths: frontend/src/app/(portal)/deviations/; backend/apps/leave_attendance/.

Satisfy: 1) Authorized wardens can record a valid student, rule, weightage and remarks; students cannot write records. 2) The saved entry includes rule ID, recorded-by user and timestamp. 3) The total equals the sum of active entries; deactivated entries are excluded. 4) Students cannot read another student's deviations; changing a rule does not alter stored historical weightage.

Use the implementation descriptions and SRS mappings in Section 1. Return a small reviewed slice with tests for these conditions; identify unmet dependencies rather than inventing records, API credentials or passing evidence.

## 6. EP-03 prompt — Study-Room Status and Laundry Tracking

Assigned owner: Aryan Singh. Paste this after the common instructions.

Implement EP-03 — Study-Room Status and Laundry Tracking for HostelSpace. Work one story at a time in dependency order, not the entire epic in one unreviewable change.

### US-09 — Display and update study-room Open or Closed status | Owner: Aryan

Paths: frontend/src/app/(portal)/study-room/; backend/apps/facilities/.

Satisfy: 1) Only Open or Closed is accepted as the saved status. 2) Students see the latest saved status, optional timing/message and update time. 3) Only permitted staff can update it; the updating user is stored. 4) No booking button, reservation API or student status-edit action is provided.

### US-10 — Assign and view the allocated laundry day | Owner: Aryan

Paths: frontend/src/app/(portal)/laundry/; backend/apps/facilities/.

Satisfy: 1) Laundry Staff can assign or update one valid weekday for a student. 2) The student sees the assigned day or a clear Not assigned message. 3) A student cannot select, reserve or modify a laundry day through the page or API. 4) An allocation update preserves earlier handover and collection records.

### US-11 — Record clothes handed over to laundry | Owner: Aryan

Paths: frontend/src/app/(portal)/laundry/handover/; backend/apps/facilities/.

Satisfy: 1) A valid student, non-empty item details and positive whole-number count create a Received record. 2) The record stores the server handover time and receiving staff member. 3) Missing fields, zero count and negative count are rejected. 4) Only Laundry Staff can create the record; the student can read only personal records.

### US-12 — Record collection and retain laundry history | Owner: Aryan

Paths: frontend/src/app/(portal)/laundry/; backend/apps/facilities/.

Satisfy: 1) Collection is allowed only for an existing Received record. 2) The record becomes Collected and keeps both handover and collection timestamps. 3) A repeated collection request does not create another record or overwrite the original completion time. 4) Students cannot change status or read another student's history.

Use the implementation descriptions and SRS mappings in Section 1. Return a small reviewed slice with tests for these conditions; identify unmet dependencies rather than inventing records, API credentials or passing evidence.

## 6. EP-04 prompt — Complaints, Mess and Hostel Fees

Assigned owner: Atiksh Gour. Paste this after the common instructions.

Implement EP-04 — Complaints, Mess and Hostel Fees for HostelSpace. Work one story at a time in dependency order, not the entire epic in one unreviewable change.

### US-13 — Submit and track a maintenance complaint | Owner: Atiksh

Paths: frontend/src/app/(portal)/complaints/; backend/apps/student_services/.

Satisfy: 1) A valid room, category and non-empty description create an Open complaint with an ID. 2) Missing or invalid input produces field-level errors and no complaint. 3) The student can view their own complaint and latest status after refresh. 4) Another student's complaint cannot be read or modified.

### US-14 — Assign complaints and update maintenance progress | Owner: Atiksh

Paths: frontend/src/app/(portal)/complaints/manage/; backend/apps/student_services/.

Satisfy: 1) The Director can assign a complaint to an active Maintenance Staff account. 2) Only assigned maintenance staff or the Director can update its permitted status transitions. 3) Every transition stores actor, time and remarks; a resolved complaint cannot silently revert. 4) The student sees the latest status but cannot edit assignment or staff remarks.

### US-15 — Manage mess allocations and active coupons | Owner: Atiksh

Paths: frontend/src/app/(portal)/mess/; backend/apps/student_services/.

Satisfy: 1) Only Mess Staff can create or update allocations and coupons. 2) A student sees the current assigned mess and only active, currently valid coupons. 3) An invalid validity range or missing coupon code is rejected. 4) Students cannot edit records or view another student's allocation.

### US-16 — View fees and verify a test payment result | Owner: Atiksh

Paths: frontend/src/app/(portal)/fees/; backend/apps/student_services/payments.py.

Satisfy: 1) A student can view personal fees and initiate an attempt for the server-calculated due amount. 2) Only a verified successful result matching fee, amount and currency marks that fee Paid. 3) Failed or cancelled attempts leave the fee Unpaid; an unverified result remains Pending. 4) A unique reference and transaction-safe update prevent duplicate success records. 5) Passwords, card details and UPI PINs are never collected or stored by HostelSpace; no real charge is made in tests.

Use the implementation descriptions and SRS mappings in Section 1. Return a small reviewed slice with tests for these conditions; identify unmet dependencies rather than inventing records, API credentials or passing evidence.

## 6. EP-05 prompt — Architecture, Setup and Automated Delivery

Assigned owner: Shared. Paste this after the common instructions.

Implement EP-05 — Architecture, Setup and Automated Delivery for HostelSpace. Work one story at a time in dependency order, not the entire epic in one unreviewable change. Each person must select only their named story below.

### US-17 — Baseline architecture and scaffold the monorepo | Owner: Anvi

Paths: frontend/src/app/; backend/config/; docs/03-architecture/; docs/decisions/.

Satisfy: 1) The repository contains working Next.js and Django/DRF skeletons plus a successful health response. 2) Architecture and deployment diagrams identify browser, frontend, API, database and external payment/location interfaces. 3) The document change log records stack updates and the duplicate UC-02 correction proposal without changing FR IDs. 4) All four members review the architecture and can run the initial skeleton.

### US-18 — Agree data models, permissions and API contracts | Owner: Arin

Paths: backend/config/; docs/04-design/; docs/04-design/api-contract.md.

Satisfy: 1) The design covers profiles, notices/rules, study status, laundry, leave, sessions/attendance, complaints, mess/coupons, fee dues/attempts and deviations. 2) The endpoint contract defines input, response, role access and validation errors for each FR. 3) Migrations apply to fresh SQLite and PostgreSQL databases; attendance and payment uniqueness are verified on PostgreSQL. 4) All four owners approve schema ownership and the role-permission matrix before merging shared models.

### US-19 — Provide a reproducible Docker development environment | Owner: Aryan

Paths: frontend/Dockerfile; backend/Dockerfile; compose.yaml; backend/apps/accounts/management/commands/.

Satisfy: 1) A clean clone starts frontend, backend and PostgreSQL using the README steps and Docker Compose. 2) Database data survives container recreation through a named volume. 3) Real secrets, SQLite files and generated build folders are excluded from Git and container build contexts. 4) Each member verifies login to a fictional demo account; demo seeding cannot run in production by default.

### US-20 — Gate pull requests and deploy reviewed releases | Owner: Atiksh

Paths: .github/workflows/; scripts/deploy.sh; scripts/smoke.sh.

Satisfy: 1) Pull requests run required checks without receiving deployment secrets. 2) A failed required job blocks merging once branch rules are configured. 3) Only a successful main push can deploy staging; failed staging smoke tests block production. 4) Production uses a configured approval gate where available and deploys the same reviewed source SHA. 5) Evidence contains run links, test reports and deployment version; no job reports success merely because a deploy was queued.

Use the implementation descriptions and SRS mappings in Section 1. Return a small reviewed slice with tests for these conditions; identify unmet dependencies rather than inventing records, API credentials or passing evidence.

## 6. EP-06 prompt — Validation, Maintenance and Final Handoff

Assigned owner: Shared. Paste this after the common instructions.

Implement EP-06 — Validation, Maintenance and Final Handoff for HostelSpace. Work one story at a time in dependency order, not the entire epic in one unreviewable change. Each person must select only their named story below.

### US-21 — Validate usability and assemble the final handoff | Owner: Anvi

Paths: docs/08-final-report/; docs/09-presentation/; tests/system/.

Satisfy: 1) Chrome, Edge and Firefox checks cover labelled forms, field errors and usable phone/desktop layouts. 2) The final report links all FRs to implemented stories, PRs, tests and validation evidence. 3) The presentation fits 15 minutes and includes a demonstration by each member. 4) Outstanding defects and limitations are listed honestly; no unfinished story is labelled implemented.

### US-22 — Maintain validation cases and verify access rules | Owner: Arin

Paths: docs/05-validation/; tests/security/; backend/apps/leave_attendance/tests/.

Satisfy: 1) Every FR, NFR and SEC ID has a validation case with preconditions, steps and expected result. 2) Protected student records reject access by another student and by unrelated staff roles in seeded tests. 3) The attendance boundary and retry cases pass on PostgreSQL without retaining raw location history. 4) Failed cases create linked Jira bugs and are retested; actual-result and evidence fields are filled only after execution.

### US-23 — Run integrated workflows and reliability checks | Owner: Aryan

Paths: tests/e2e/; tests/system/; docs/06-testing/.

Satisfy: 1) Critical end-to-end flows pass against the integrated Next.js and Django staging release. 2) Attendance retries and payment-success retries do not create duplicate records in PostgreSQL tests. 3) The proposed NFR-01 check uses 20 signed-in simulated users and 100 warm-page visits, measuring navigation-to-usable-page time with p95 at most 3 seconds; obtain faculty/team approval first. 4) The report states browser, environment, dataset, run time and observed failures; a failed target is not reported as passed.

### US-24 — Verify release recovery and complete maintenance guidance | Owner: Atiksh

Paths: docs/07-maintenance/; scripts/; backend/apps/student_services/tests/.

Satisfy: 1) A test backup restores successfully into a separate database and expected record counts match. 2) A staging rollback identifies and restores a known-good frontend/backend release; schema compatibility is checked first. 3) Payment acceptance cases pass using provider test mode and no live funds or private student data. 4) The maintenance plan names owners, weekly dependency review, backup steps and incident/bug reporting steps.

Use the implementation descriptions and SRS mappings in Section 1. Return a small reviewed slice with tests for these conditions; identify unmet dependencies rather than inventing records, API credentials or passing evidence.

# 7. Timeline / Build Sequence

Proposed plan: Monday 7 September–Sunday 18 October 2026. Release target: Friday 16 October, with 17–18 October held for recovery or submission preparation. No actual faculty deadline was supplied, so completion before that unknown deadline cannot be promised. Replace these dates with the official deadline before starting the Jira sprints.

Capacity assumption: approximately 8–10 focused hours per member per week, plus brief planning/review time. That is a planning assumption, not known availability or an estimate derived from story points. Re-estimate after week one; do not cut security checks, testing or evidence to preserve an unrealistic date.

| Week / dates | Planned work and owner | Exit gate |
|---|---|---|
| W1 · 7–13 Sep | Anvi: US-17 architecture/skeleton and US-01 auth slice. Arin: US-18 schema/API contracts and initial US-22 validation design. Aryan: US-19 Docker and seed data. Atiksh: US-20 initial CI, staging setup and provider test access. | All run the stack; auth/session proxy works; CI genuinely tests a vertical slice. First staging deployment and architecture/design review recorded. |
| W2 · 14–20 Sep | Anvi: US-02 profile, US-03 information/rules. Arin: US-05 leave, US-06 warden/Chief workflow. Aryan: US-09 study status, US-10 assigned day. Atiksh: US-13 complaints, US-14 staff progress. | One integrated student/staff increment. Permission and invalid-input tests pass; incomplete stories remain open. |
| W3 · 21–27 Sep | Anvi: US-04 Director controls/summaries as source modules land. Arin: US-07 attendance, US-08 deviations. Aryan: US-11 received, US-12 collected. Atiksh: US-15 mess, US-16 provider test payments; complete US-20 promotion gates. | Feature-complete staging candidate, all FRs represented, no unverified payment success or duplicate accepted attendance. |
| W4 · 28 Sep–4 Oct | Anvi: US-21 usability/form fixes. Arin: US-22 access, privacy and validation execution. Aryan: US-23 integrated/browser/performance checks. Atiksh: payment edge cases and deployment version/readiness checks. | System validation results recorded; critical defects fixed or release blocked. Coverage/scanner findings documented, not hidden. |
| W5 · 5–11 Oct | Anvi: report and slides. Arin: validation retest/RTM. Aryan: regression and clean Docker run. Atiksh: US-24 backup/restore, rollback and maintenance. All: peer reviews and module explanations. | Release candidate on staging; recovery tested; genuine Jira, PR, test and container evidence assembled. |
| W6 · 12–18 Oct | 12–13 Oct: complete rehearsal and fix defects. 14 Oct: final regression/security/payment check. 15 Oct: freeze accepted documents/code. 16 Oct: approve/tag/deploy release. 17–18 Oct: reserved buffer. | All FR acceptance cases pass or limitations are explicitly accepted; every member demonstrates owned functionality. Submit only according to the faculty's actual instructions. |

## 7. Dependency order and weekly working routine

```
US-17 skeleton/architecture -> US-18 schema/API contracts
              |                          |
              +----> US-19 Docker <------+
              +----> US-01 auth -> every protected feature
US-17/18/19 -> US-20 CI and staging -> production gates
US-05 leave -> US-06 decisions/escalations
US-03 rules -> US-08 deviation records
US-10 allocation -> US-11 received -> US-12 collected
US-13 complaint -> US-14 maintenance progress
US-06/07/14/16 -> US-04 complete Director summaries
All feature slices -> US-22/23 validation and regression
US-16/20/23 -> US-24 recovery and release checks
US-21 report/demo integrates evidence throughout, not only at the end
```

| Cadence | What actually happens |
|---|---|
| Monday planning | Review remaining capacity, select small stories/subtasks, clarify acceptance criteria and dependencies. Do not move every story into the active sprint. |
| Working days | Implement and test one coherent slice; push genuine progress after the session. Record blockers promptly. See Section 5 for dated commit examples by person. |
| PR reviews | Use the reviewer rotation within one working day where possible. Recheck affected module contracts and migration changes. |
| Friday review | Demonstrate accepted stories on staging, update the RTM/test evidence, close only Done work and note defects. |
| Sprint retrospective | Record one process improvement and replan unfinished work. Preserve meaningful Jira history rather than deleting late work. |
| Release gate | Confirm tested SHA, migrations, secrets, staging result, recovery procedure and independent approval before production. |

## If the actual deadline is earlier

Replan the schedule before coding: reduce optional styling, extensive report graphics and nonessential automation first. Do not quietly remove required hostel functions, ownership, Docker/course evidence, authorization, payment verification or validation. Split large stories and record an agreed scope change with faculty if the remaining time cannot support the baseline. A real deadline needs a fresh capacity check, not fabricated calendar certainty.

## Final handoff checklist

• Accepted SRS corrections, architecture, detailed design, validation specification and current RTM.

• Source, migrations and startup instructions; individual commits and independently reviewed PRs.

• Working test-mode demonstration, CI/CD runs, Docker evidence, test/coverage/static/security reports and honest limitations.

• Maintenance and recovery guide, final report, 15-minute slides and each member’s complete-function demonstration.

## 7. References and final planning notes

Project sources: the supplied PES UE24CS341A mini-project guidelines; the Coffee Kiosk submission example (format and traceability guidance only); the prepared Team 11 HostelSpace Synopsis dated 1 September 2026; the final SRS dated 2 September 2026; and subsequent team decisions in this conversation. The Drive folder was not newly accessed for this plan.

### [R1] Next.js — Jest testing

Jest/React Testing Library scope and async Server Component limitation.

https://nextjs.org/docs/app/guides/testing/jest

### [R2] GitHub — Deployment environments

Environment approval, secret and branch protection concepts; verify account/visibility availability.

https://docs.github.com/en/actions/concepts/workflows-and-actions/deployment-environments

### [R3] Render — Deploys

CI-dependent deployment behavior and why it must not bypass a staging gate.

https://render.com/docs/deploys

### [R4] Vercel — GitHub Actions example

CLI-based frontend deployment and environment credential pattern.

https://github.com/vercel/examples/tree/main/ci-cd/github-actions

Public documentation checked while preparing this plan on 5 September 2026. Provider plans, action/runtime support and package patches can change; verify during setup. These references explain tools, not faculty requirements.

## What this document does and does not claim

This is a development plan and ready-to-copy backlog, not an implementation, an imported Jira board, a deployed application or a completed validation report. Story estimates, acceptance refinements, dates and hosting choices are proposals for team agreement. It preserves the four members' functional ownership and the eleven SRS functional requirements while documenting the additional work needed to deliver and demonstrate them.

Next action: agree the schedule and open decisions, create Jira and GitHub using individual accounts, then start US-17–US-20 and the first authenticated vertical slice. Update the existing Synopsis/SRS through a recorded revision; do not present this plan as if those PDF files were already changed.